

Projet42 - Cahier des charges

Cahier des charges

Auteur: Gilbert robert@prolibre.com - 2026-05-20

Contexte

Les environnements OpenStack ou Cloud (VPS) deviennent rapidement complexes à administrer lorsque plusieurs projets, réseaux, instances et équipes coexistent.

Les besoins récurrents sont :

- disposer d'une vision centralisée des ressources ;
- connaître l'état et la consommation des serveurs ;
- automatiser le déploiement d'infrastructures ;
- standardiser le provisionnement ;
- superviser les performances et les coûts ;
- disposer d'un portail simple pour les équipes techniques.

Ce projet vise à développer une plateforme unifiée permettant :

- l'inventaire des ressources OpenStack ;
 - le déploiement d'infrastructures "Infrastructure as Code" ;
 - le provisionnement automatisé ;
 - la supervision et les métriques temps réel.
-

Objectifs du projet

L'application devra permettre

La gestion et visualisation OpenStack

- connexion à une ou plusieurs plateformes OpenStack ;
- récupération des projets (tenants) ;
- visualisation des ressources - affichage des consommations CPU/RAM/stockage ;
- état des machines (RUNNING, STOPPED, ERROR).

Déploiement d'infrastructure (IaC)

- déploiement automatisé d'environnements ;
- utilisation d'OpenTofu ;
- gestion de templates d'infrastructure ;
- versionnement Git ;
- lancement de plans/apply ;
- gestion des variables et secrets.

Périmètre fonctionnel

Le Dashboard devra afficher :

Vue infrastructure

- nombre total de VM ;
- ressources consommées ;
- répartition par projet ;
- état global des instances ;
- disponibilité des hyperviseurs.

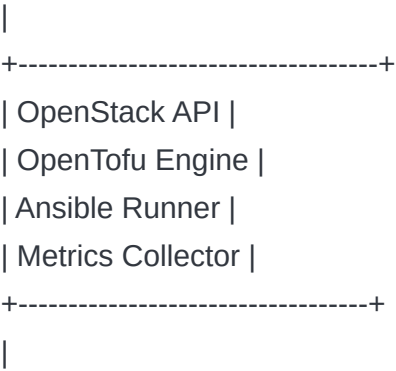
Vue projet

- quotas utilisés ;
- coûts estimés ;
- inventaire des ressources ;
- topologie réseau.

Vue déploiements

- derniers déploiements ;
- historique ;
- succès/échecs ;
- logs d'exécution.

Utilisateur -> Frontend Web -> Backend API



Base de données

|

Grafana / Prometheus

Stack technique proposée

Ce sont des indications. S'il y a mieux ou plus adapté il faut l'étudier

Frontend

- React / Next.js
- TypeScript
- TailwindCSS
- Material UI

Backend

- Python FastAPI
ou
- Node.js NestJS

Infrastructure as Code: OpenTofu

Provisionnement: Ansible

Supervision

- Prometheus
- Grafana

Base de données: PostgreSQL

CI/CD: GitLab / Gitea

Conteneurisation: Docker

Fonctionnalités détaillées

Inventaire OpenStack

Fonctionnalités

- découverte automatique des ressources ;
- synchronisation périodique ;
- recherche et filtrage ;
- export CSV/JSON ;
- multi-région.

Déploiement Infrastructure as Code

Fonctionnalités

- catalogue de templates ;
- création d'environnements ;
- génération automatique de variables ;
- prévisualisation du plan ;
- rollback.

Exemple de workflow

- Sélection d'un template
- Saisie des variables
- Génération OpenTofu
- Validation du plan
- Déploiement
- Provisionnement Ansible
- Monitoring automatique

Provisionnement Ansible

Fonctionnalités

- lancement de playbooks ;
- gestion des rôles ;
- exécution parallèle ;
- logs temps réel ;
- reprise sur erreur.

Monitoring & observabilité

Collecte des métriques

Infrastructure

- CPU
- RAM
- disque
- réseau

OpenStack

- nombre d'instances ;

- consommation projet ;
- quotas ;
- état hyperviseurs.

Dashboards Grafana

- capacité cluster ;
- santé infrastructure ;
- disponibilité ;
- alertes ;
- tendances.

Alerting

- email ;
- webhook;
- mattermost.

Déploiement de la plateforme

Docker Compose

Services :

- frontend
- backend
- postgres
- prometheus
- grafana
- ansible-runner

Roadmap projet

Phase 1 — MVP

Durée estimée : 6 semaines

- authentification ;
- connexion OpenStack ;
- inventaire ressources ;
- dashboard basique.

Phase 2 — IaC

Durée : 4 semaines

- intégration OpenTofu ;
- gestion templates ;
- déploiements.

Phase 3 — Provisionnement

Durée : 3 semaines

- intégration Ansible ;
- inventaires dynamiques ;
- exécution playbooks.

Phase 4 — Monitoring

Durée : 3 semaines

- Prometheus ;
- Grafana ;
- Alerting.

Phase 5 — Industrialisation

Durée : 4 semaines

- sécurité ;
- CI/CD ;
- documentation.

Livrables

Techniques

- code source ;
- documentation ;
- API Swagger/OpenAPI ;
- playbooks Ansible ;
- templates OpenTofu ;

- dashboards Grafana.

Exploitation

- guide d'installation ;
 - guide utilisateur ;
 - procédures de sauvegarde ;
 - supervision.
-

Bénéfices attendus

Techniques

- automatisation ;
- réduction des erreurs ;
- standardisation ;
- visibilité globale.

Opérationnels

- gain de temps ;
 - déploiements rapides ;
 - supervision centralisée ;
 - meilleure gouvernance.
-

Évolutions futures ou pas!

- support multi-cloud ;
 - FinOps ;
 - autoscaling ;
 - catalogue de services ;
 - IA pour optimisation des ressources ;
 - remédiation automatique.
-

Conclusion

Cette plateforme permettra de centraliser :

- la gestion OpenStack ;
- le déploiement d'infrastructure ;
- le provisionnement automatisé ;
- la supervision temps réel.

L'objectif est de fournir un portail moderne de type "Cloud Management Platform" simplifiant l'exploitation des environnements OpenStack tout en appliquant les principes DevOps et Infrastructure as Code.